Question Difficulty Classification System

System Architecture Design Document

Team Veda

Narendra Singh (2401010292)

Lead Architect & Developer

Gaurav Meena

Data Engineer

Keshav

Data Analyst

NST Computer Science and AI
Rishihood University

March 2, 2026

Executive Summary

This document provides a comprehensive architectural overview of the Question Difficulty Classification System. The system implements a modular, scalable machine learning pipeline for automatically categorizing educational questions into Easy, Medium, and Hard difficulty levels. The architecture follows industry best practices with clear separation of concerns, ensuring maintainability, reproducibility, and ease of deployment.

Key Architectural Features:

- Modular pipeline architecture with independent components
- Data leakage prevention through proper separation
- Scalable feature engineering using TF-IDF vectorization
- Production-ready model serialization and deployment
- Comprehensive evaluation and monitoring framework

Contents

Executive Summary **2**

1 System Overview **5**

2 High-Level Architecture **5**

3 Layered Architecture **6**

4 Detailed Component Architecture **6**

4.1 Data Layer Architecture 6

4.1.1 Data Layer Specifications 7

4.2 Preprocessing Layer Architecture 7

4.3 Feature Engineering Layer 8

4.4 Model Layer Architecture 8

5 Data Flow Architecture **9**

6 Technical Stack Architecture **10**

7 Module Specifications **10**

7.1 Data Processing Module 10

7.2 Text Preprocessing Module 11

7.3 Feature Engineering Module 12

7.4 Model Training Module 12

7.5 Evaluation Module 13

7.6 Prediction Module 14

8 Deployment Architecture **15**

8.1 Serialization Format 15

9 Performance Architecture **16**

9.1 Computational Complexity 16

9.2 Memory Footprint 17

10 Security Architecture **17**

11 Monitoring Architecture **17**

11.1 Key Metrics 17

List of Figures

1 High-Level System Architecture 6
3 Data Layer Components 7
4 Text Preprocessing Pipeline 7
5 Feature Engineering Components 8
6 Model Training Architecture 8
7 Complete Data Flow Diagram 9
8 Deployment Architecture 15
9 Monitoring Architecture 17

List of Tables

1 Architectural Quality Attributes 5
2 Data Layer Component Specifications 7
3 Complete Technology Stack 10
4 Computational Complexity Analysis 16
5 Memory Usage Estimation (10,000 samples) 17

1 System Overview

The Question Difficulty Classification System is designed as a modular, end-to-end machine learning pipeline that processes educational questions and predicts their difficulty levels. The architecture emphasizes six key quality attributes:

Table 1: Architectural Quality Attributes

Attribute	**Description**
Modularity	Independent, replaceable components with well-defined interfaces
Scalability	Ability to handle increasing data volumes without redesign
Reproducibility	Consistent results across different runs and environments
Maintainability	Clean separation of concerns for easy updates and debugging
Extensibility	Easy addition of new features, models, or data sources
Reliability	Robust error handling and validation at each pipeline stage

2 High-Level Architecture

Figure 1 illustrates the high-level system architecture, showing the main components and data flow through the pipeline.

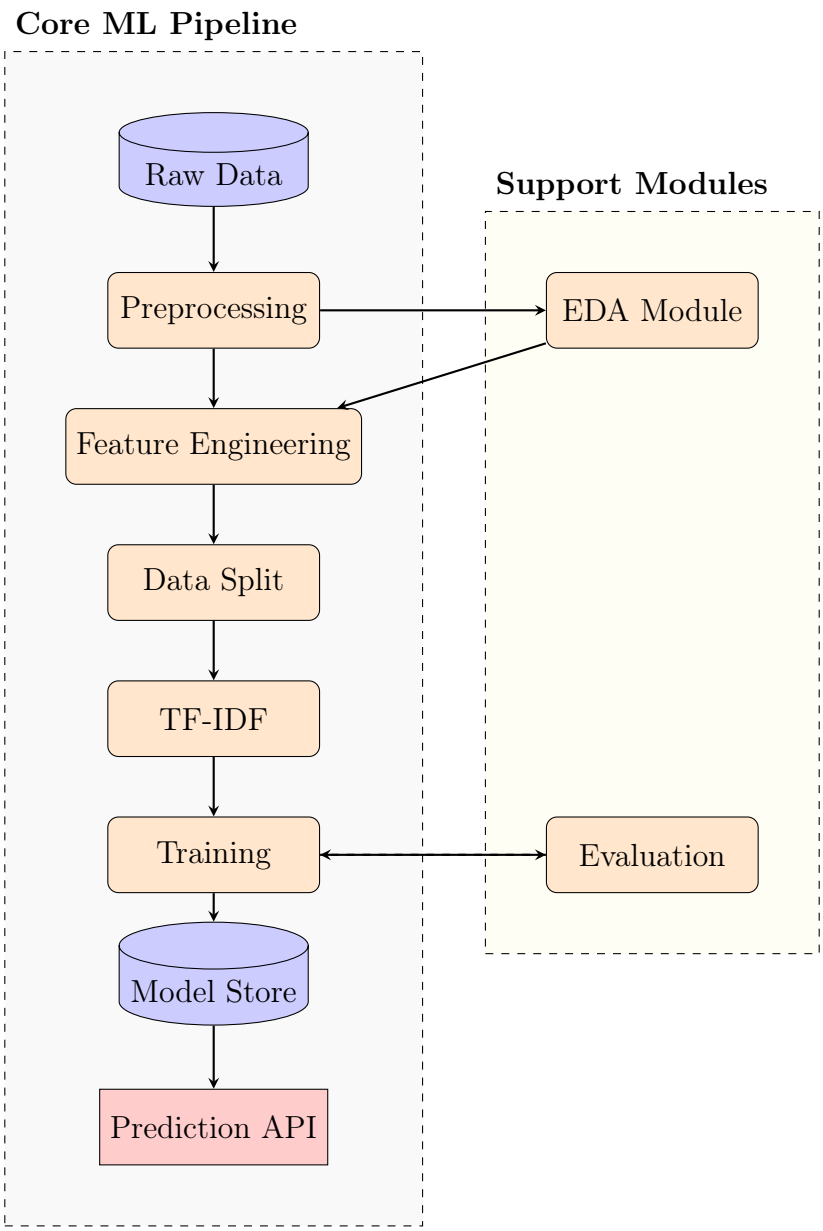

Figure 1: High-Level System Architecture

3 Layered Architecture

The system follows a layered architecture pattern with four distinct layers, as shown in Figure **??**.

4 Detailed Component Architecture

4.1 Data Layer Architecture

The Data Layer handles all data ingestion, validation, and storage operations.

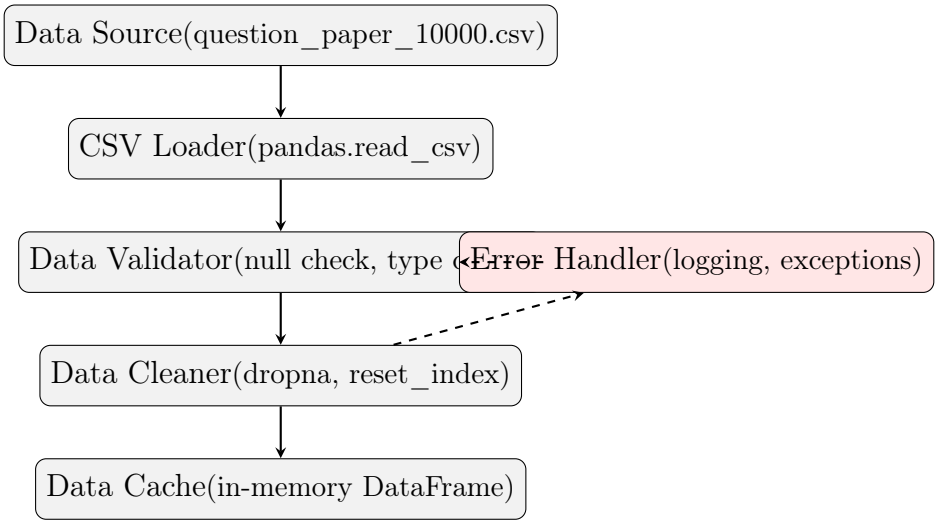

Figure 3: Data Layer Components

4.1.1 Data Layer Specifications

Table 2: Data Layer Component Specifications

Component	**Input**	**Output**
CSV Loader	Raw CSV file	Raw DataFrame
Data Validator	Raw DataFrame	Validation report
Data Cleaner	Validated DataFrame	Clean DataFrame
Data Cache	Clean DataFrame	In-memory storage
Error Handler	Exception	Log entry

4.2 Preprocessing Layer Architecture

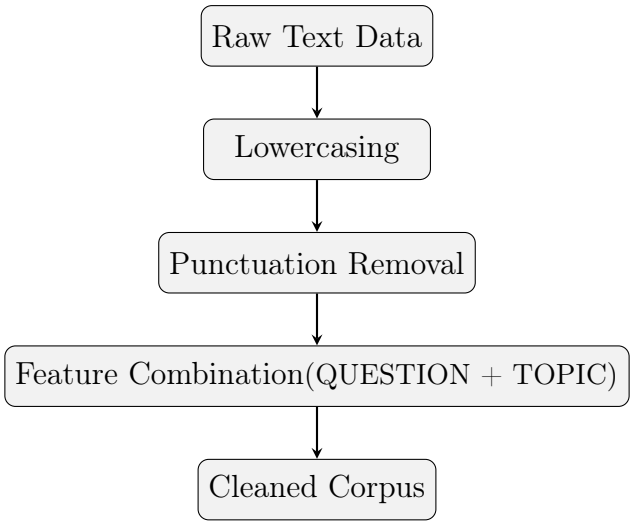

Figure 4: Text Preprocessing Pipeline

4.3 Feature Engineering Layer

The feature engineering component transforms text into numerical representations using TF-IDF vectorization:

$$\text{tfidf}_{t,d} = \underbrace{\text{tf}_{t,d}}_{\text{term frequency}} \times \underbrace{\log\left(\frac{N}{\text{df}_t}\right)}_{\text{inverse document frequency}} \tag{1}$$

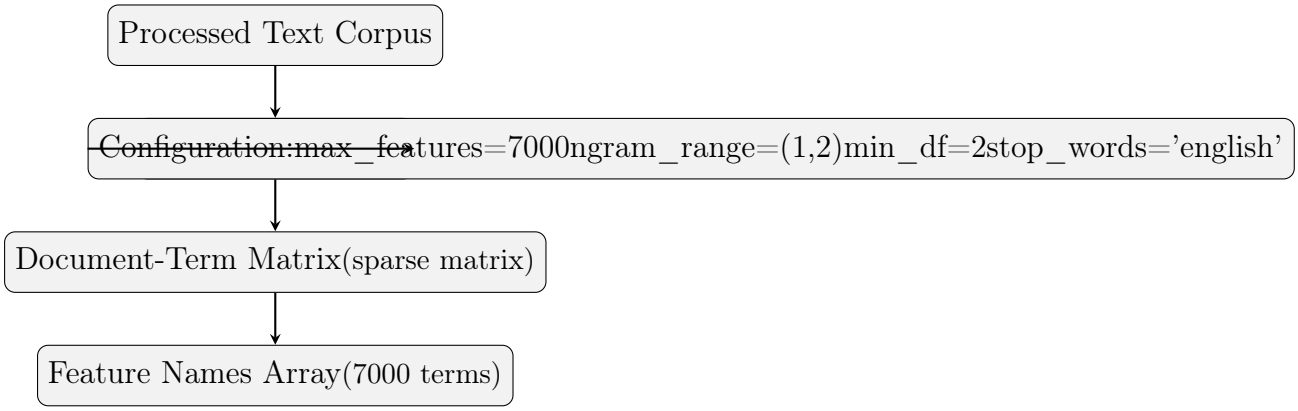

Figure 5: Feature Engineering Components

4.4 Model Layer Architecture

The model layer implements Logistic Regression for multi-class classification:

$$P(y = k|\mathbf{x}) = \frac{e^{\beta_k^T \mathbf{x}}}{\sum_{j=1}^{K} e^{\beta_j^T \mathbf{x}}} \tag{2}$$

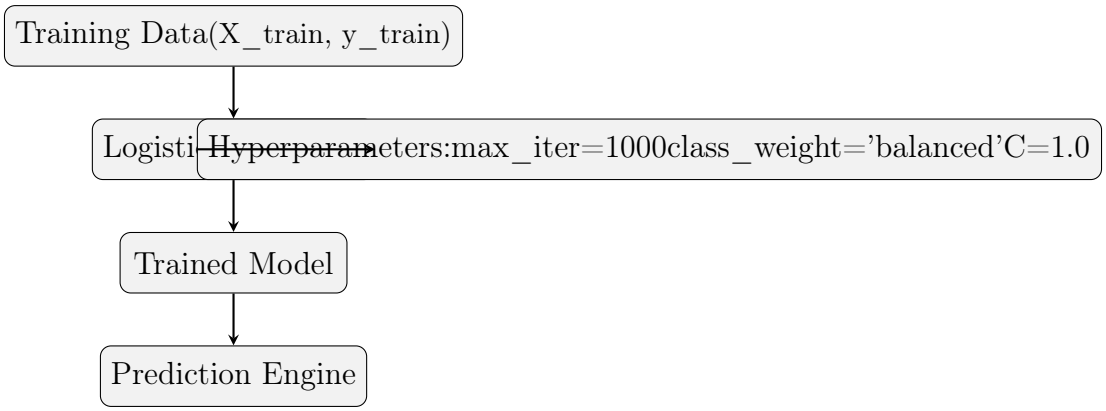

Figure 6: Model Training Architecture

5 Data Flow Architecture

Figure 7 illustrates the complete data flow through the system.

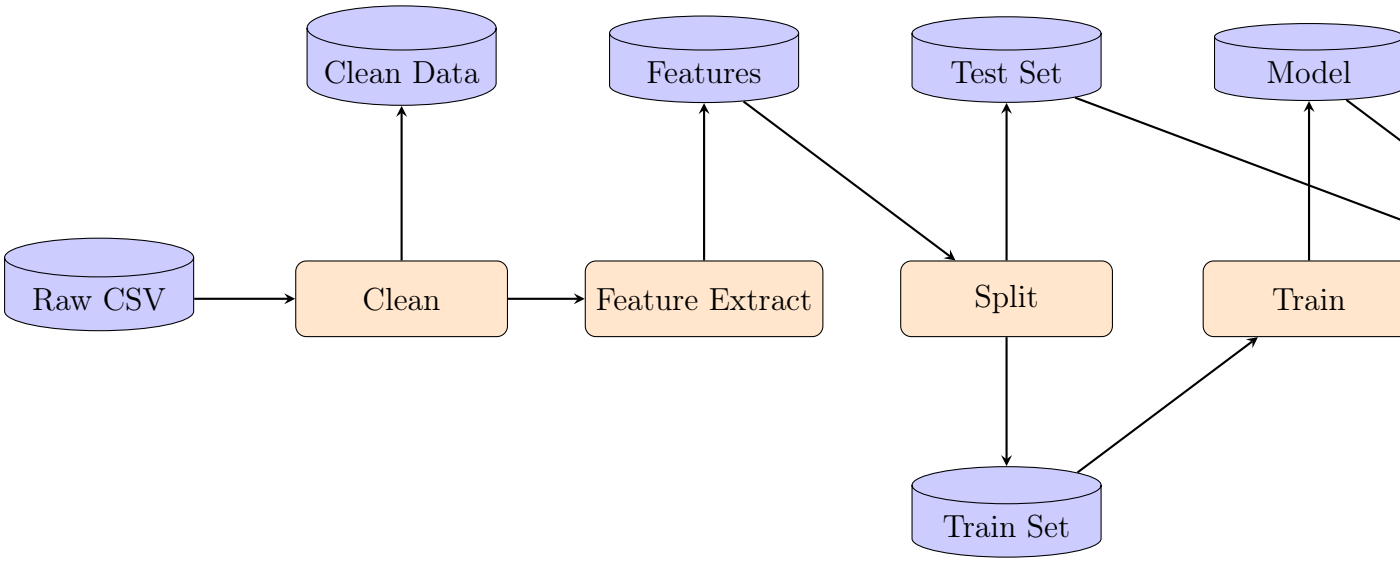

Figure 7: Complete Data Flow Diagram

6 Technical Stack Architecture

Table 3: Complete Technology Stack

Layer	Technology	Version	Purpose
Programming Languages			
	Python	3.8+	Core development
	LaTeX	-	Documentation
Data Processing			
	Pandas	1.3+	Data manipulation
	NumPy	1.21+	Numerical operations
	CSV	-	Data storage
Natural Language Processing			
	scikit-learn TF-IDF	1.0+	Text vectorization
	Regex	-	Text cleaning
	NLTK (stopwords)	3.6+	Stop word removal
Machine Learning			
	scikit-learn	1.0+	ML algorithms
	LogisticRegression	-	Classification
	LinearSVC	-	Alternative model
	cross_validation	-	Model validation
Visualization			
	Matplotlib	3.4+	Plotting
	Seaborn	0.11+	Statistical viz
Development Tools			
	Jupyter/Colab	- | Interactive development	
	Git	2.3+	Version control
	GitHub	-	Code hosting
Deployment			
	Pickle	- | Model serialization	
	Joblib	- | Alternative serialization	

7 Module Specifications

7.1 Data Processing Module

Listing 1: Data Processing Module

```python
class DataProcessor:
    """Handles all data loading and cleaning operations"""

    def __init__(self, data_path):
```

```python
        self.data_path = data_path
        self.df = None

    def load_data(self):
        """Load CSV with error handling"""
        self.df = pd.read_csv(
            self.data_path,
            engine='python',
            quotechar='"',
            on_bad_lines='skip'
        )
        return self

    def clean_data(self):
        """Remove missing values and reset index"""
        self.df = self.df.dropna()
        self.df.reset_index(drop=True, inplace=True)
        return self

    def validate_data(self):
        """Check data quality"""
        assert self.df.isnull().sum().sum() == 0
        return self
```

7.2 Text Preprocessing Module

Listing 2: Text Preprocessing Module

```python
import re

class TextPreprocessor:
    """Handles all text cleaning and preprocessing"""

    @staticmethod
    def clean_text(text):
        """Lowercase and remove punctuation"""
        text = str(text).lower()
        text = re.sub(r"[^\w\s]", "", text)
        return text

    def preprocess_dataframe(self, df):
        """Apply cleaning to all text columns"""
        df["QUESTION_TEXT"] = df["QUESTION_TEXT"].apply(self.clean_text)
        df["TOPIC"] = df["TOPIC"].apply(self.clean_text)
        df["ANSWER_TEXT"] = df["ANSWER_TEXT"].apply(self.clean_text)
        return df

```

```
    def combine_features(self, df):
        """Combine question text with topic"""
        df["COMBINED_TEXT"] = (
            df["QUESTION_TEXT"] + " " + df["TOPIC"]
        )
        return df
```

7.3 Feature Engineering Module

Listing 3: Feature Engineering Module

```
from sklearn.feature_extraction.text import TfidfVectorizer

class FeatureExtractor:
    """Handles TF-IDF vectorization"""

    def __init__(self, max_features=7000, ngram_range=(1,2)):
        self.vectorizer = TfidfVectorizer(
            max_features=max_features,
            stop_words='english',
            ngram_range=ngram_range,
            min_df=2
        )
        self.fitted = False

    def fit_transform(self, texts):
        """Fit vectorizer and transform texts"""
        self.vectorizer.fit(texts)
        self.fitted = True
        return self.vectorizer.transform(texts)

    def transform(self, texts):
        """Transform using fitted vectorizer"""
        assert self.fitted, "Vectorizer not fitted yet"
        return self.vectorizer.transform(texts)

    def get_feature_names(self):
        """Get vocabulary terms"""
        return self.vectorizer.get_feature_names_out()
```

7.4 Model Training Module

Listing 4: Model Training Module

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

class ModelTrainer:
```

```python
    """Handles model training and evaluation"""

    def __init__(self, test_size=0.2, random_state=42):
        self.test_size = test_size
        self.random_state = random_state
        self.model = None

    def split_data(self, X, y):
        """Stratified train-test split"""
        return train_test_split(
            X, y,
            test_size=self.test_size,
            random_state=self.random_state,
            stratify=y
        )

    def train(self, X_train, y_train):
        """Train logistic regression model"""
        self.model = LogisticRegression(
            max_iter=1000,
            class_weight='balanced'
        )
        self.model.fit(X_train, y_train)
        return self.model

    def predict(self, X):
        """Make predictions"""
        assert self.model is not None, "Model not trained"
        return self.model.predict(X)
```

7.5 Evaluation Module

Listing 5: Evaluation Module

```python
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score,
    confusion_matrix, classification_report
)

class ModelEvaluator:
    """Handles comprehensive model evaluation"""

    @staticmethod
    def evaluate(y_true, y_pred):
        """Compute all metrics"""
        return {
            'accuracy': accuracy_score(y_true, y_pred),
            'precision_macro': precision_score(
```

```
                y_true, y_pred, average='macro'
            ),
            'recall_macro': recall_score(
                y_true, y_pred, average='macro'
            ),
            'f1_macro': f1_score(
                y_true, y_pred, average='macro'
            ),
            'confusion_matrix': confusion_matrix(y_true, y_pred),
            'classification_report': classification_report(
                y_true, y_pred
            )
        }

    @staticmethod
    def plot_confusion_matrix(cm, labels):
        """Visualize confusion matrix"""
        import seaborn as sns
        import matplotlib.pyplot as plt

        plt.figure(figsize=(8,6))
        sns.heatmap(
            cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=labels, yticklabels=labels
        )
        plt.title('Confusion Matrix')
        plt.xlabel('Predicted')
        plt.ylabel('Actual')
        plt.show()
```

7.6 Prediction Module

Listing 6: Prediction Module

```
class DifficultyPredictor:
    """End-to-end prediction pipeline"""

    def __init__(self, model, vectorizer, preprocessor):
        self.model = model
        self.vectorizer = vectorizer
        self.preprocessor = preprocessor

    def predict(self, question_text, topic=""):
        """Predict difficulty for a single question"""
        # Preprocess
        cleaned_q = self.preprocessor.clean_text(question_text)
        cleaned_t = self.preprocessor.clean_text(topic)
        combined = cleaned_q + " " + cleaned_t

```

```
        # Vectorize
        vectorized = self.vectorizer.transform([combined])

        # Predict
        prediction = self.model.predict(vectorized)
        return prediction[0]

    def predict_batch(self, questions, topics=None):
        """Predict for multiple questions"""
        if topics is None:
            topics = [""] * len(questions)

        results = []
        for q, t in zip(questions, topics):
            results.append(self.predict(q, t))
        return results
```

8 Deployment Architecture

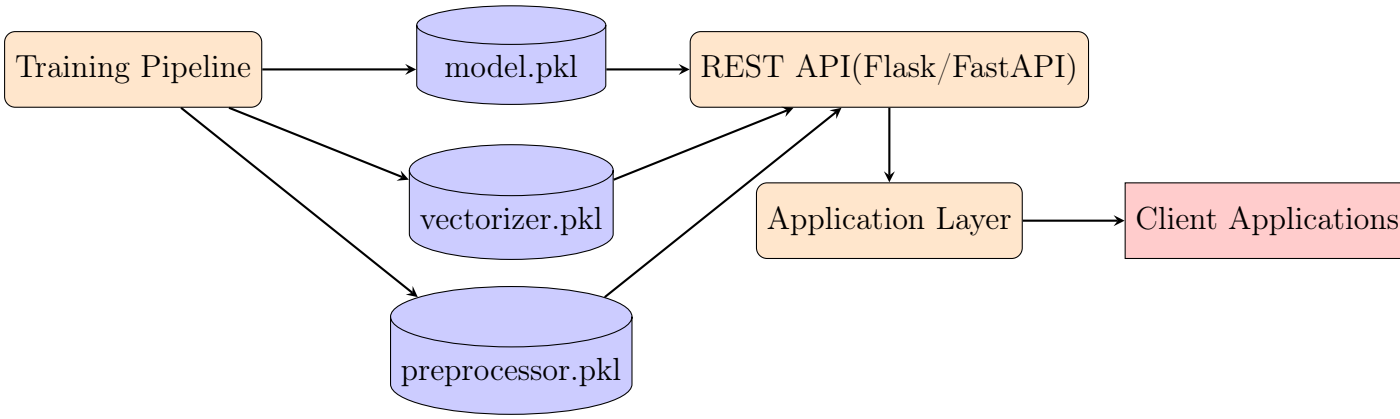

Figure 8: Deployment Architecture

8.1 Serialization Format

Listing 7: Model Persistence

```
import pickle

# Save artifacts
def save_artifacts(model, vectorizer, preprocessor, path='./
    artifacts/'):
    with open(f'{path}model.pkl', 'wb') as f:
        pickle.dump(model, f)

    with open(f'{path}vectorizer.pkl', 'wb') as f:
```

```
        pickle.dump(vectorizer, f)

    with open(f'{path}preprocessor.pkl', 'wb') as f:
        pickle.dump(preprocessor, f)

# Load artifacts
def load_artifacts(path='./artifacts/'):
    with open(f'{path}model.pkl', 'rb') as f:
        model = pickle.load(f)

    with open(f'{path}vectorizer.pkl', 'rb') as f:
        vectorizer = pickle.load(f)

    with open(f'{path}preprocessor.pkl', 'rb') as f:
        preprocessor = pickle.load(f)

    return model, vectorizer, preprocessor
```

9 Performance Architecture

9.1 Computational Complexity

Table 4: Computational Complexity Analysis

Operation	**Time Complexity**	**Space Complexity**
Data Loading	O(n)	O(n)
Text Preprocessing	O(n $\cdot l$)	O(n $\cdot l$)
TF-IDF Fitting	O(n $\cdot m \cdot d$)	O(m $\cdot d$)
TF-IDF Transform	O(n $\cdot m$)	O(n $\cdot m$)
Model Training	O(k $\cdot n \cdot f$)	O(k $\cdot f$)
Model Prediction	O(k $\cdot f$)	O(1)

n = samples, l = text length, m = vocabulary size,
d = documents, f = features, k = classes

9.2 Memory Footprint

Table 5: Memory Usage Estimation (10,000 samples)

Component	Memory Estimate
Raw DataFrame	5-10 MB
Cleaned DataFrame	5-10 MB
TF-IDF Sparse Matrix	50-100 MB
Feature Names (7000 terms)	0.5 MB
Model Coefficients	0.2 MB
Total	60-120 MB

10 Security Architecture

- **Data Validation**: Input sanitization to prevent injection attacks
- **Error Handling**: Graceful failure without exposing internals
- **Access Control**: API authentication for production deployment
- **Audit Logging**: Track all predictions and system access
- **Input Limits**: Maximum text length to prevent DoS attacks

11 Monitoring Architecture

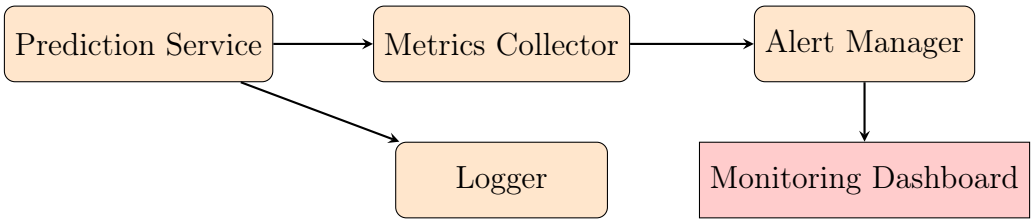

Figure 9: Monitoring Architecture

11.1 Key Metrics

- Prediction latency (p50, p95, p99)
- Throughput (predictions/second)
- Model accuracy drift
- Error rates
- Resource utilization (CPU, memory)